

D-PAD SERIAL BRIDGE - MANUAL TESTING QUICKSTART

=====

Purpose: quick reference for bringing up the D-pad -> Arduino Nano bridge for testing, without re-deriving the commands each time.

Hardware summary

- Arduino Nano (CH340 USB-serial chip) plugged into the Pi, enumerates as /dev/ttyUSB0.
- Pin mapping: Up = D3, Right = D4, Down = D5, Left = D6.
- Sketch: robot/arduino/dpad_pin_driver/dpad_pin_driver.ino

EVERY-TIME WORKFLOW

=====

1. SSH into the Pi (or open a terminal on it).
2. Go to the workspace and source it (required in every NEW terminal you open -- this does not persist across terminals):

```
cd ~/robot-code-leglift/ros2_ws
source install/setup.bash
```

3. Launch both nodes (joystick + bridge) with one command:

```
ros2 launch dpad_serial_bridge dpad_serial_bridge_test.launch.py
```

This starts joy_linux_node and dpad_serial_bridge_node together, with NO other part of the robot stack running (no controllers armed, no actuators powered) -- safe for repeated bench testing.

4. Press each D-pad direction on the PS5 controller and watch the terminal log output for warnings or errors.
5. (Optional) In a SECOND terminal, also sourced (step 2), watch the Nano's own debug echo of what it received:

```
python3 -c "
import serial, time
s = serial.Serial('/dev/ttyUSB0', 115200, timeout=1)
time.sleep(2)
while True:
    line = s.readline()
    if line:
        print(line.decode(errors='replace'), end='')
"
```

You should see lines like "OK pin 3 -> HIGH" / "OK pin 3 -> LOW" as you press and release Up, etc.

ONE-TIME SETUP (only needed after pulling new changes to this package)

=====

```
cd ~/robot-code-leglift/ros2_ws
colcon build --packages-select dpad_serial_bridge
source install/setup.bash
```

After that, the "EVERY-TIME WORKFLOW" above is all you need per session.

TROUBLESHOOTING

=====

"Package 'dpad_serial_bridge' not found"

-> You forgot to `source install/setup.bash` in this terminal. Sourcing does not carry over between terminal windows/tabs.

Nothing happens when you press D-pad directions

-> Confirm /joy is actually publishing: `ros2 topic echo /joy` in another terminal, press a direction, and see if axes[6]/axes[7] change.
-> Confirm the Nano is plugged in: `ls /dev/ttyUSB0`
-> Check dialout group membership: `groups \$USER` should include "dialout". If missing: `sudo usermod -aG dialout \$USER`, then log out and back in.

Arduino Serial Monitor shows nothing when typing commands by hand

-> Check the line-ending dropdown (bottom-right of the Serial Monitor window) is set to "Newline" or "Both NL & CR", not "No line ending".
-> Check the baud rate dropdown is set to 115200.

Unplugging the Arduino crashes the ROS node

-> This should NOT happen -- the node is designed to log a warning and keep retrying the connection every 2 seconds instead of crashing. If it does crash, that's a bug to report back, not expected behavior.

VERIFIED SO FAR (as of the last test session)

=====

- Arduino sketch bench-tested standalone via Serial Monitor: all four pins (D3/D4/D5/D6) toggle correctly on U/R/D/L commands.
- D-pad's real /joy encoding confirmed on hardware: two hat-style axes, not buttons (axes[6] = left/right, axes[7] = up/down; horizontal axis inverted from the typical convention, corrected via config).
- Full pipeline confirmed end-to-end: PS5 D-pad -> /joy -> ROS node -> serial -> Arduino -> correct LED lights up per direction.

STILL TO DO

=====

- Unplug/replug test: confirm the node logs a warning and recovers cleanly instead of crashing, and that pin state re-syncs correctly after reconnecting.
- Run the FULL robot stack (``ros2 launch neural_controller launch.py``) and confirm the bridge still works correctly alongside everything else.
- Reboot the Pi and confirm robot.service starts cleanly with the new node included, both with the Arduino plugged in and unplugged.
- Remove the temporary debug `Serial.print` lines from the .ino sketch once everything above is confirmed working (they're harmless bench-test noise but not needed for the final deployed version).